

Astant

A Retrieval-Augmented Generation (RAG) System with Memory for Regulations, Syllabus, FAQs and Notices

Project Title	Astant – RAG + Memory Assistant for Academic Information
Domain	Artificial Intelligence / Machine Learning – Agentic AI, NLP
Student Name	Thamizh Ezhilan T
Program	B.E. Computer Science and Engineering
Institution	Anna University, BIT Campus, Trichy
Duration	2024 – 2027

1. Project Overview

1.1 Project Title

Astant – RAG + Memory: An Agentic AI Assistant for Regulations, Syllabus, FAQs and Notices.

1.2 Problem Statement

Students and faculty routinely need quick answers to questions about academic regulations, syllabus content, examination rules, and institutional notices. This information is usually scattered across multiple PDFs, circulars, and web pages, making it time-consuming to search manually. Traditional keyword search fails to understand natural-language questions, often returns irrelevant documents, and has no memory of prior interactions — forcing users to repeat context in every query. There is a need for an intelligent assistant that can understand questions in plain language, retrieve the exact relevant information from institutional documents, and hold a coherent conversation across multiple turns.

1.3 Brief Description of the Project

Astant is an agentic AI assistant built on a Retrieval-Augmented Generation (RAG) pipeline combined with a persistent memory layer. It ingests institutional documents — regulations, syllabi, FAQs, and notices — converts them into vector embeddings, and stores them in a vector database. When a user asks a question, Astant retrieves the most relevant document chunks, combines them with the ongoing conversation history stored in memory, and uses a large language model (LLM) to generate an accurate, context-aware, and source-grounded answer. This reduces hallucination, keeps responses grounded in official documents, and gives the assistant continuity across a conversation.

2. Objectives & Proposed Solution

2.1 Project Objectives

- To design a document ingestion pipeline that can process regulations, syllabi, FAQs, and notices into a searchable knowledge base.
- To implement a Retrieval-Augmented Generation (RAG) pipeline that grounds LLM responses in actual institutional documents rather than the model's own memory.
- To add a conversational memory layer so the assistant can maintain context across multiple questions in a session.
- To reduce hallucinated or inaccurate answers by citing the retrieved source for every response.
- To provide students and faculty with a fast, natural-language interface for academic queries, available 24/7.

2.2 How the Agentic AI Solution Works

Astant follows a multi-step agentic workflow rather than a single LLM call:

- **Ingestion:** Source documents (PDFs/text of regulations, syllabus, FAQs, notices) are cleaned, split into smaller chunks, and converted into vector embeddings.
- **Indexing:** The embeddings are stored in a vector database for fast similarity search.
- **Query understanding:** When a user asks a question, the agent analyzes intent and reformulates the query if needed (e.g., resolving pronouns using conversation history).
- **Retrieval:** The agent searches the vector store for the top-matching document chunks relevant to the query.
- **Memory injection:** Relevant prior conversation turns are pulled from the memory store and combined with the retrieved documents.
- **Generation:** The LLM generates a final answer grounded in the retrieved context, citing the source document/section.
- **Memory update:** The new question-answer pair is stored back into memory for continuity in future turns.

2.3 Key Features

- Natural-language Q&A; over regulations, syllabus, FAQs, and notices.
- Source-grounded answers with citations to reduce hallucination.
- Persistent conversational memory across multi-turn sessions.
- Semantic search using vector embeddings instead of plain keyword matching.
- Easily extensible knowledge base — new notices/circulars can be added without retraining the model.
- Simple chat-based interface for students and faculty.

3. Implementation & Results

3.1 Technologies / Tools Used

Category	Tool / Technology
Programming Language	Python
LLM / Generation	Large Language Model API (e.g., OpenAI / Gemini / open-source LLM)
Embeddings	Text embedding model for semantic vector representation
Vector Database	FAISS / Chroma (for similarity search over document chunks)
Orchestration Framework	LangChain / LlamaIndex (RAG pipeline & agent orchestration)
Memory Store	Conversation buffer / vector-based memory for context retention
Document Processing	PDF/text parsing and chunking utilities
Interface	Web-based chat interface

Note: Replace the placeholder tools above with the exact libraries/APIs used in your implementation.

3.2 Working Process

- Collect and organize source documents (regulations, syllabus copies, FAQ sheets, notices).
- Pre-process and chunk the documents into manageable text segments.
- Generate vector embeddings for each chunk and store them in the vector database.
- Build the retrieval pipeline to fetch the most relevant chunks for a given query.
- Integrate a memory module to retain prior conversation context.
- Connect the retrieval + memory output to the LLM to generate the final grounded response.
- Test the assistant with sample student/faculty queries and refine prompts and chunking strategy.

3.3 Screenshots / Output

[Insert screenshots of the chat interface, sample query-response pairs, and the retrieval pipeline output here before submission.]

3.4 Results Achieved

[Add measured outcomes here, for example: number of documents indexed, average response time, accuracy/relevance of retrieved answers based on manual evaluation, or user feedback collected during testing.]

4. Conclusion & Future Scope

4.1 Project Conclusion

Astant demonstrates how a Retrieval-Augmented Generation pipeline combined with a persistent memory layer can turn scattered institutional documents into an accessible, conversational knowledge assistant. By grounding responses in retrieved source material rather than relying solely on the language model's internal knowledge, the system provides more accurate and trustworthy answers to regulation, syllabus, FAQ, and notice-related queries, while the memory layer allows for natural, multi-turn conversations.

4.2 Challenges Faced

- *[Add specific challenges, e.g., handling inconsistent document formats, choosing an optimal chunk size for retrieval accuracy, managing context window limits, or tuning the memory module to avoid irrelevant recall.]*

4.3 Future Enhancements

- Support for voice-based queries in addition to text.
- Automatic ingestion of new notices/circulars as they are published, without manual re-indexing.
- Multi-language support for regional language queries.
- Role-based responses (student vs. faculty view) for more personalized answers.
- Integration with the institution's official portal/app for wider accessibility.
- Feedback loop where users can rate answer accuracy to continuously improve retrieval quality.

5. References

- *[Add references to research papers, framework documentation (e.g., LangChain/LlamaIndex docs), vector database documentation, and LLM provider documentation used during development.]*